Ship & Found: Five Assessment Tasks

Read this part before you start anything

There are five tasks. They run in order, because each one builds on the last. By the end you will have one application, one repository, one commit history spanning several weeks, and a documentation set covering every topic these tasks touch, from the design system through to account deletion.

That repository is the point. It is what you send to someone who is deciding whether to hire you.

Everything here comes from what we have covered together so far. Nothing has been added that you have not already been taught. If a step refers to something you cannot remember, the topic is named at the top of each task. Go back to the recording before you ask.

We are still building Spotter. These tasks cover the ground we have got through up to now, which is why they stop where they stop. More is coming, and what comes next will not be tested here.

The rule that shapes every task

You may not build a gym.

We are building Spotter together, in front of you, session by session. A model that has read our session recordings, or that is handed the Spotter repository as it currently stands, can reproduce it. That makes a gym application worthless as an assessment of you.

So you pick a different business. It must have the same shape as Spotter, which means all four of these:

1. Real members or customers who log in.
2. Money moving, or a subscription that expires.
3. Records the business keeps about each person.
4. Questions those people ask repeatedly that only this business can answer.

Examples that fit: a driving school, a private clinic, a tailoring business with measurements on file, a school with fee records, a cooperative society, a short-let apartment manager, a laundry service with pickup records, an equipment hire business, a church or mosque with membership records, a cold room with distributor accounts.

Examples that do not fit: a to-do app, a blog, a weather app, a portfolio site, a calculator, anything with no accounts and no money.

Every validation rule, every retention decision, every rate limit and every table in your schema has to be derived from your business. You cannot copy the six-digit member number, the seven-day grace period or the two-minute expiry from our sessions, because none of those numbers were derived for your business. If your submission contains a gym member number, you have copied rather than reasoned, and the task is failed.

How these tasks are built, and why

You will use AI agents to do this work. That is the whole point of this programme and nobody is asking you to code without one. What these tasks will not accept is work you cannot account for.

So the evidence you submit is evidence an agent cannot produce, because it came off your machine while you were working. There are nine mechanisms and you should understand all of them before you start:

1. Machine-specific evidence. You submit terminal output containing your own port numbers, your own process IDs, your own local addresses, your own timestamps, and your own database row IDs. A model cannot invent your process ID 14229. If two students submit the same PID, both are failed.

2. Deliberate breakage. Several steps require you to break something on purpose, capture the broken state, then fix it and capture the fix. The broken state is specific to your code. A model asked for a broken state produces a generic one.

3. Commit history as process evidence. Minimum commit counts are set per task, and the commits must be spread across at least three different days. One commit containing everything is evidence that the work was not done the way it is described.

4. Your own numbers, defended. Where we chose a value together in class, you choose your own and defend it. "Why is your code expiry eleven minutes" has no correct answer. It has an answer that is yours, or it has no answer.

5. Wrong-answer analysis. Each task gives you a plausible but incorrect answer and asks you to say what is wrong with it. Models tend to agree with fluent text. A student who has understood the mechanism does not.

6. Cross-task consistency. Task five requires you to cite specific values from your own tasks one to four. Figures that do not match your own earlier submissions are a fail.

7. Primary research. One task requires you to talk to a real person running a real business and report what they actually said. Not a persona. A person, with a name and a business you can point at.

8. Your own defects. Each task requires you to name a real defect in your own code, by file and line number, and say what it would cost in production. A perfect submission with no defects named is marked down, because a project with nothing wrong with it was either trivial or is being misrepresented.

9. The viva. Every task is defended out loud. The follow-up questions are chosen after reading your submission, so they are about your decisions and not about the topic in general. You cannot prepare answers to questions that do not exist until your work is read.

Recordings are optional, and they carry extra marks

Each task names a short recording you can submit. Screen shared, your own voice, walking through something specific.

None of them are compulsory. Your bandwidth, your data cost and your equipment are not being assessed. A task submitted without any recording can still score full marks on everything else.

But each recording carries up to five bonus marks on top of the task score, because talking through your own work out loud is the single best preparation for the interview you are doing all this for, and because it is the fastest way for anyone reviewing your submission to see that you understand it.

Three ways to take those marks, in descending value:

Screen and voice together. Up to five marks. Any tool, any quality. A phone pointed at your laptop counts.

Voice only, with the relevant screenshots in your evidence folder. Up to three marks. Useful if your connection cannot carry video.

A written walkthrough in your own words, timestamped against your evidence files. Up to two marks. Slowest to read and still better than nothing.

Where a step says to record something as proof, such as a screen reader reading your form, you can submit a written transcript of what it announced instead. The transcript is accepted in full. The audio is the bonus.

The interview questions in each task

Each task ends with interview questions. They are not made up. Most were taken from published interview guides and question sets for AI engineer, AI agentic engineer, forward deployed engineer, full stack engineer and design engineer roles, and the ones written specially for these tasks are marked as such. Every source is listed at the end of this document.

Do not memorise answers. The point is that every question here is one you could be asked in the next three months, and most of them you can already answer because you are about to build the thing they are asking about.

What you submit, and where

One public GitHub repository per task, or one repository with five top-level folders. Your choice, stated in your main readme.

Required structure:

your-repo/ README.md task-01-design-system/ README.md evidence/ docs/ task-02-repository-and-scaffold/ task-03-client-validation/ task-04-server-security/ task-05-sessions-compliance-defence/

Inside every task folder:

README.md. What this task was, what you built, what you decided, what you got wrong. Written for a technical interviewer who has four minutes.

evidence/. Screenshots, terminal output saved as text files, screen recordings, photographs of hand-drawn work. Named so the name says what it shows: evidence/03-port-conflict-before-kill.txt, not evidence/screenshot1.png.

docs/. Your written work. The decision log, the interview answers, the retention matrix, and the other written deliverables named in each task.

Terminal output is submitted as text files, not screenshots of text. Text can be searched and compared between students. Screenshots of text cannot, which is exactly why some people prefer them.

The repository readme, which is itself assessed

Your main readme is the first thing anyone sees, and in a hiring process it is often the only thing they see. A repository showing only folders and files tells a technical interviewer nothing, and reading your code to work out what the project is should not be their job.

Your main readme must contain, in this order:

1. **What this is.** One sentence. The business, the person it serves, the one thing that must work.
2. **Status.** What is finished, what is half done, what is not started. Honest. A readme claiming everything works, attached to an application that does not, is worse than no readme.
3. **The stack, with reasons.** Not a logo wall. Each choice and one line on why it, rather than the alternative you considered.
4. **Getting started.** The exact commands, in order, that take someone from a fresh clone to a running application. Someone who has never seen your project must be able to follow them without asking you anything.
5. **Environment variables.** The names and what each is for. No values. Never values.
6. **What this deliberately does not do.** The scope boundary, with reasons.
7. **Known defects.** The real ones, with file references.

Section seven is the one that separates submissions. Most people delete it. Keep it.

Timeline and marking

Each task has a working window. They run in order, because each builds on the last. Tasks are marked on a four-part split, the same for all five:

Working software, 30 percent. It runs, and it does what you say it does.

Evidence, 20 percent. The machine-specific proof is present, legible and matches your account.

Reasoning, 35 percent. The written work. Why, not what. This is the largest share deliberately.

Defence, 15 percent. The viva.

On top of that, up to five bonus marks for the optional walkthrough, so a task is scored out of 105.

A task where the software works and the reasoning is thin scores lower than one where the software is partly finished and every decision is understood. That split is intentional, and it matches how these roles are actually screened.

Automatic fail conditions, on any task:

- A gym, or a member number, or any value lifted from our sessions without deriving it for your own business.
- Terminal output that does not come from your machine.
- A readme claiming functionality that does not exist.
- A secret committed to the repository at any point in its history, including a commit later reverted. History is searched.
- No defects named.

- Inability to explain your own code line by line in the viva.

TASK ONE: THE DESIGN SYSTEM, AND PROVING THE AGENT USED IT

What this draws on: the typography and shadow systems, the token export, and the design half of information design.

Window: five days.

What this task tests

Whether you can build a design system, get it into a form an agent consumes, and then prove the agent actually consumed it rather than assuming it did.

Why it matters

Three of you showed your projects in our session and all three looked identical, because all three asked an agent to make it look good and the agent produced the most likely interface. With nothing supplied, the most likely interface is the correct output for an unspecified request.

A design system is the input that stops that. It is an engineering artifact, not a design preference, and for design engineering roles it is the job. One posting for a design engineer asked explicitly for fluency in Figma variables and tokens and for bridging Figma to code as a single source of truth.

The scenario

Daniel is a brand designer. He hands a UI designer one brand colour and a logo. Three weeks later he opens the product and his brand colour appears twice, in a footer and on one button. Everything else is the default palette the system generated.

Nobody did anything wrong. The brand colour was one input among many and the system diluted it.

That scenario is the reason for the brand override step below, and it is the difference between having a colour and having a system built around that colour.

Steps

1. Choose your business and write the brief

Before any colour, write docs/business-brief.md, no more than 400 words:

- The business, named. Real or realistic, your choice, but specific.
- The person who uses it. Age range, device, connection quality, how technical.
- The one thing that must work.
- Three questions this business gets asked repeatedly that only it can answer.

- Why the interface needs to feel a particular way, and what way.

The last point drives your typography. A clinic and a short-let apartment manager do not get the same type.

2. Build the colour system

In Figma. Not in code, and not from a generator you cannot explain.

1. Pick your brand colour. State in writing where it came from.
2. Pin it as primary 40 so the palette generates around it instead of absorbing it. Capture the palette before and after pinning, as two images, and write one paragraph on the visible difference.
3. Generate the tonal palettes. State how many tones and why that number is enough for light and dark from one source.
4. Add functional colours for error, warning and success.
5. Define your colour roles. Surface, on-surface, primary, on-primary, and whatever else your interface needs. A role is a job, not a colour.
6. Check contrast on every role pair you will actually use. Record the ratio for each. Any pair below the threshold is either fixed or documented as not used for text, with a reason.

3. Build the typography system

Five roles, three sizes each. Display, headline, title, body, label.

For every one of the fifteen styles record: font family, weight, size, letter spacing, line height.

Three constraints:

Line height follows the formula. Font size times 1.4, 1.5 or 1.6. Say which multiplier you used for which role and why it differs between roles, if it does.

Letter spacing is judged visually, not copied. Compact spacing improves scannability on web and mobile, where people scan rather than read. But different typefaces occupy different amounts of space, so the right value for DM Sans is not the right value for your font. You must capture two screenshots of your display large style, one at the default spacing and one at your chosen spacing, and write one paragraph on what changed.

You must diverge from the documented default at least twice, deliberately. The Material baseline recommends particular values. We diverged on font family, weight and letter spacing when we set up the Spotter scale. You will diverge somewhere too. For each divergence write: what the default was, what you chose, why, and in what situation your choice would be the wrong one.

That last clause is the part that cannot be generated. A model will tell you why your choice is good. Saying where it fails requires you to understand the trade-off.

4. Build the shadow system

Three shadows, soft, medium and hard, built from your primary colour rather than from black. Record x offset, y offset, blur, spread and opacity for each.

Write two sentences on why a shadow built from the brand colour reads differently from one built from black.

5. Export, and catch the omission

Export colours, typography and effects as JSON.

The export will omit your colour system unless you explicitly include Figma variables. This happened to us in the session and the file had to be deleted and re-exported.

Deliverable: both files. Submit evidence/tokens-incomplete.json, the export missing the colour variables, and evidence/tokens-complete.json, the correct one. Then write the one-line diff: what was missing, and how you noticed.

Submitting only the correct file means you either did not make the mistake or you hid it. Either way you lose the marks for this step. Make the mistake on purpose if you have to.

6. Convert JSON to CSS variables, by instructing an agent

JSON gets the system into the project where the agent can read it. CSS is what the browser applies. The conversion is a script, and you instruct the agent to write it.

Your instruction must specify that colour roles are used on the interface and primitive colours are not, so the design system stays the single source of truth.

Submit the prompt you used, verbatim, in docs/prompts.md. Including the parts that did not work and what you changed.

7. Prove the agent consumed it

This is the step most people skip. In our own session the system was built and the agent ignored it, because the agent never received it.

Build one screen with the agent. Any screen. Then:

1. Search your entire codebase for raw colour values. Submit the search command you ran and its full output, even if the output is empty. Especially if it is empty.
2. Ask the agent, in writing, which typography style it used for a specific element. Record its answer. In class the agent admitted it had used the wrong style when asked directly.
3. Submit before and after screenshots of that screen: once built without the system, once with it.

8. The deliberate regression

Create a branch. On it, break the token referencing: point a colour role at a raw hex value instead of at the palette.

Then change your brand colour at the source.

Capture what happens. The parts still referencing the palette follow. The part you broke does not.

Submit a screenshot of the inconsistent interface and three sentences on why this is the whole argument for token referencing. Then delete the branch without merging it, and show the git log proving it was

never merged.

Deliverables

task-01-design-system/ README.md docs/ business-brief.md type-scale.md all 15 styles, every value colour-roles.md roles, what each points at, contrast ratios shadows.md divergences.md each divergence, why, and where it fails prompts.md verbatim, including failed attempts defects.md interview-answers.md evidence/ 01-palette-before-pin.png 02-palette-after-pin.png 03-spacing-default.png 04-spacing-chosen.png 05-tokens-incomplete.json 06-tokens-complete.json 07-raw-value-search.txt the command and its full output 08-agent-typography-answer.txt 09-screen-without-system.png 10-screen-with-system.png 11-broken-referencing.png 12-git-log-branch-unmerged.txt 13-walkthrough.mp4 optional, bonus marks

Optional walkthrough, up to five bonus marks. Four minutes, your voice, screen shared. Walk through your type scale and justify three values. Not the whole scale. Three values you pick, and you say why those three were the hardest to settle. Voice only or a written walkthrough both count for fewer marks, as set out at the front of this document.

The interview defence

Answer all twelve in docs/interview-answers.md. Two hundred words maximum each.

Questions one to eight are published interview questions for design engineer, frontend and accessibility roles, taken verbatim or near-verbatim.

1. How would you structure design tokens?
2. When is a token change breaking?
3. How would you support multiple brands and themes?
4. Can tokens guarantee accessible contrast?
5. How do you ensure that colour contrast guidelines are met within a website or application's design?
6. Can you discuss the differences between WCAG levels A, AA and AAA?
7. Discuss the importance of semantic HTML in the context of accessibility.
8. How do you handle design handoff to developers using Figma?

Questions nine to twelve were written for this assessment, because no published interview set covers this ground. That gap is itself worth knowing: the industry has not caught up to asking these, and a candidate who can answer them is answering something most interviewers have not thought to ask yet.

9. Why do AI-generated interfaces tend to look generic, and what specifically do you hand an agent to stop it?
10. Your line height for display large is a specific number. Why that number and not one step either side of it? What would you lose with each?
11. Point at one place in your own token structure that you would build differently if you started again. Say what you would do instead and what it would cost.

12. **Wrong-answer analysis.** A candidate says: "I set up design tokens by exporting the hex values from Figma into a CSS file, so every colour in the codebase comes from one place. That makes it a single source of truth." This sounds correct and is not. Say precisely what is missing and what will go wrong in month four.

How this is graded

- The system is complete and internally consistent: 25
- Evidence is machine-specific and matches your account: 20
- Divergences and contrast ratios are reasoned, not asserted: 20
- The proof that the agent consumed the system: 20
- Viva: 15
- Optional walkthrough: up to 5 bonus marks on top of the 100

Automatic fail

- A palette generated by a tool you cannot explain.
- Only the correct JSON export submitted.
- An empty raw-value search with no command shown.
- Contrast ratios asserted without numbers.

TASK TWO: SCAFFOLD IT, SHIP THE REPOSITORY, THEN FIND OUT WHAT IS ACTUALLY RUNNING

What this draws on: scaffolding, the development server, database diagnostics, information design, and the working tree.

Window: six days.

What this task tests

Whether you can take an application from nothing using an agent, present the repository so a hiring manager can assess it in four minutes, and then diagnose what is actually running rather than trusting what the interface shows you.

Why it matters

Scaffolding is where most people hand control to the agent and never get it back. The agent writes continuously without deliberating, and leaves unused, broken and duplicate functions behind, which degrades performance. Scaffolding from context files is what reduces that.

The diagnosis half matters more. In our own session the signup flow looked complete. The logs showed a 401 then a 200 and no 201 anywhere. A 201 is what signals a record being created. Prisma Studio returned "no database URL found". The application was presenting login and signup visually with nothing stored behind it.

An engineer who cannot tell the difference between an interface that works and an application that works is not adding anything, and this is the single most common failure in AI-built software.

The scenario

Aisha is hired to review a product a founder built in six weeks with an agent. The founder shows her the signup flow, the dashboard, and a settings page. Everything clicks through. The founder wants to know why conversion is zero.

Aisha opens the server logs. There is not a single 201 in four weeks. Nobody has ever been able to create an account. Every visitor who tried to sign up saw a success screen and became nothing.

That demonstrates the gap between a rendered interface and a working application, which is what step six of this task makes you find in your own code.

Steps

1. Write the scaffolding prompt yourself

Not an agent writing the prompt. You write it, then you may use an agent to tighten it, and you submit both versions.

The prompt must instruct the coding agent to:

1. Read all context files and design system files before writing any code.
2. Summarise its understanding and wait for your approval.
3. Treat those files as the only source of truth.
4. Ask rather than assume when two files disagree.
5. Use design system tokens with no hardcoded styling values.
6. Build only the project setup and a minimal home screen. Not the product.
7. List every file it created and every assumption it made.
8. Audit its own response against these instructions.

Point six is the one people drop. Building only the foundation first stops the agent attempting the whole product in one pass. Scaffolding takes thirty to forty-five minutes when done properly.

Submit docs/scaffold-prompt.md with your first draft, your final version, and a paragraph on what changed between them and why.

2. Scaffold, and capture the plan before you approve it

Run the prompt. Before approving, capture the agent's plan and its stated assumptions as a text file.

If the agent listed zero assumptions, say in writing whether you believe that, and what you checked to decide.

Submit evidence/01-agent-plan.txt and evidence/02-files-created.txt.

3. Choose your models deliberately

Antigravity's agent is slower because it reads, analyses and plans before calling a tool. Open Code is optimised for writing code quickly rather than planning.

Use Antigravity for the scaffold. Use something else for at least one later feature. Then write docs/model-choices.md: which model for which job, what you observed about the difference, and one task where you chose wrong and switched.

4. Get it running, and read the logs

Start the development server. A static live server renders static HTML only and cannot run a dynamic application, so if that is what you reached for, record that you did.

Capture, as text files:

1. The terminal output showing the server starting, including your port number.
2. The first page load, with the HTTP status and the load time in milliseconds.

Then answer in docs/performance-note.md: your first load time, how far it is from a reasonable target, and what you think accounts for the gap. Ours took 10,220 milliseconds on first load and dropped to 790 on reload, against a target nearer 10.

5. Break the server three ways and recover

Three deliberate failures. Capture the terminal output for each, then the fix.

The nested folder. Agents sometimes create a project folder inside your project folder, so commands run from the wrong place. Reproduce it or recreate the conditions, capture the error, then fix it both ways: by telling the agent to rescaffold in place, and by changing directory. Submit both.

The port conflict. Start a second process on the same port. Capture the conflict. Find the process ID holding the port, submit the command you used to find it, end it, and capture the server then starting cleanly. Your process ID will not match anyone else's.

A 404 and a blank page. Cause one. Capture the status code and the time. Then diagnose it rather than immediately prompting the agent. Write what you checked, in order, before you asked for help.

Submit all six files, three failures and three recoveries, named so the pairing is obvious.

6. Find out whether anything is actually being stored

The core of this task.

Add a form that writes to a database. A signup, a booking, an enquiry, whatever your business needs.

Then prove it writes, and prove it in a way that cannot be faked:

1. Submit the form. Capture the full log output for that request.
2. Find the status codes. Name them. If there is no 201 anywhere, say so and explain what that means.
3. Open your database inspector. Capture the table with your row in it, including the row ID and the timestamp.
4. Submit the same form a second time with different values. Capture the second row. Two rows with your own timestamps minutes apart.

Then the inverse, which is the part that matters:

5. Deliberately break the database connection. Rename the environment variable, or point the URL at nothing.
6. Submit the form again. Capture what the interface shows the user, and capture the logs.
7. Write docs/false-success.md: what the user saw, what actually happened, and how a person reviewing this product in a demo would have been fooled.

7. The repository, as a professional artifact

Commit and push. Create the repository if it does not exist.

Then the readme, to the specification in the front matter of this document. All seven sections.

Test it for real: give the readme to someone who has never seen your project and ask them to get it running. Record what they got stuck on, verbatim, in docs/readme-test.md, and then fix the readme. A readme nobody has tested is a guess.

8. The working tree, drawn by hand

The working tree is the pipeline from your computer, to Git's warehouse on your computer, to GitHub, to production. A branch holds it together.

Draw it. On paper, by hand. Label: your computer, the local Git store, GitHub, production, the main branch, and two feature branches. Mark the point where code becomes visible to users, and mark the point past which a mistake can no longer be undone quietly.

Photograph it with today's date written on the page in your handwriting. Submit as evidence/14-working-tree-drawing.jpg.

Then write docs/working-tree.md: what breaks when a tree has one branch, and why that matters more after your first deployment than before it.

9. Branch properly, and prove it

After your first push, leave main.

1. Create a branch named for the work, and check out to it.
2. Do the whole of step ten on that branch.
3. Commit and push the branch without merging.
4. Open a pull request against your own main branch and write a real description.

5. Merge it.
6. Then create a second branch, make a change you decide against, and delete the branch without merging.

Submit evidence/15-git-log-graph.txt showing the branch structure, and the pull request URL in your readme.

Your commit history must show at least twelve commits across at least three different days, with messages that say what changed. "Updates" and "fixes" are not messages.

10. Information design, which is the half nobody does

Your application has a head and a body. Configuration lives in the head. Content lives in the body.

The head. Add all five, and record the before and after:

1. A page title that changes with navigation, not one static string.
2. A short description, a tagline, under ten words.
3. SEO tags on your marketing page only, after reading them rather than accepting them.
4. A favicon you made.
5. Open Graph tags, so a shared link produces a preview rather than a bare URL.

Verify each one. Submit a screenshot of the rendered head on two different routes, proving the title actually changed.

The body. Apply information design to one screen:

1. Remove something that is there and should not be. Say what and why. In Spotter we took out a text label the adjacent logo already said, which invited redirection confusion.
2. Collapse a choice the user should not have to make. In Spotter we collapsed two buttons into one, because a new visitor should not have to choose between login and signup before they know what the product is. Record the growth reasoning.
3. Set a maximum width on your main heading so it wraps where you want it to, creating a deliberate shape that directs attention. Capture it wrapping correctly and wrapping badly.
4. Size your buttons with padding in rem units, not fixed width and height. State your padding and corner radius and why.
5. Apply your own label style from task one to every button, then check the agent actually did it by asking it directly.

Interaction and responsiveness.

6. Every clickable element gets pointer cursor behaviour and visible hover feedback. Your logo links home.
7. Scale the browser down until your largest text breaks. Capture it broken. Then make it responsive with clamp, and capture it working at three widths.

Deliverables

task-02-repository-and-scaffold/ README.md docs/ scaffold-prompt.md model-choices.md performance-note.md false-success.md readme-test.md working-tree.md information-design.md defects.md interview-answers.md evidence/ 01-agent-plan.txt 02-files-created.txt 03-server-start.txt 04-first-load.txt 05-nested-folder-error.txt 06-nested-folder-fixed.txt 07-port-conflict.txt 08-port-conflict-resolved.txt 09-404-error.txt 10-404-resolved.txt 11-write-succeeded-logs.txt 12-database-rows.png 13-write-failed-logs.txt 14-working-tree-drawing.jpg 15-git-log-graph.txt 16-head-route-one.png 17-head-route-two.png 18-heading-wrap-bad.png 19-heading-wrap-good.png 20-responsive-three-widths.png 21-walkthrough.mp4 optional, bonus marks

Optional walkthrough, up to five bonus marks. Five minutes. Open your server logs, submit a form, and talk through every status code as it appears. Unscripted. If you cannot record video, the same thing as a voice note with your log files alongside it still earns marks.

The interview defence

Answer all fifteen in docs/interview-answers.md. Two hundred words each.

Version control. Questions one to seven are published interview questions, taken verbatim or near-verbatim. These are asked constantly and answered badly.

1. What is the difference between Git Flow and trunk-based development, and why have a lot of teams moved toward trunk-based?
2. What does git push --force actually do, and why is --force-with-lease generally safer?
3. Why is rewriting history, through rebase, amend or force push, dangerous on a shared branch?
4. What is the difference between git merge and git rebase, and when would you actually pick one over the other?
5. What causes a merge conflict, and what is actually happening in the file when you see the conflict markers?
6. What is the difference between the working directory, the staging area and the repository?
7. What is branch protection, and what does it typically enforce?

The role and the engagement. Published questions from forward deployed engineer question sets.

8. Walk me through a technical project you built end to end.
9. Tell me about a time when something you built broke in production.
10. How do you approach learning a new technical platform or codebase quickly when starting a new engagement?
11. What is the difference between code that is correct and code that is complete?
12. How do you design software for a customer who will need to maintain it after you are gone?

Then three about your work, written for this assessment:

13. Your first page load took a specific number of milliseconds. Where did that time go? Name the parts you can measure and the parts you are guessing about.
14. Your application reported success to the user while writing nothing to the database. Explain the mechanism that allowed that, and name the one check that would have caught it in thirty seconds.

15. **Wrong-answer analysis.** A candidate says: "I always check my app is working by signing up a test user and confirming I get taken to the dashboard. If the dashboard loads, the backend is working." Say what is wrong with this, what it would miss, and what you would do instead.

Also be ready for this one in the viva, which is published and which a surprising number of candidates have had to answer for real: how would you handle a situation where you accidentally committed sensitive information to a public repository?

How this is graded

- Scaffolding done from context files, with the plan captured before approval: 15
- The three breakages, with machine-specific recovery evidence: 20
- The false success investigation and write-up: 25
- Readme quality, tested on a real person: 15
- Information design and responsiveness: 10
- Viva: 15
- Optional walkthrough: up to 5 bonus marks on top of the 100

Automatic fail

- Fewer than twelve commits, or commits on fewer than three days.
- A process ID matching another student's.
- No captured failure state for the database break.
- A readme that has not been tested on anyone.
- Any secret in the git history.

TASK THREE: THE FORM, ATTACKED FROM BOTH SIDES

What this draws on: single page architecture, client-side validation, and accessible error states.

Window: five days.

What this task tests

Whether you can build a registration form that collects correct data, is usable by someone who cannot see colour, and whose client-side checks you understand to be a convenience rather than a security control.

Why it matters

Forms are where almost every application touches the public, and they are consistently the weakest part of AI-built software. The agent produces a form that works for a cooperative user and fails for everyone else: bots, agents filling fields with single tokens, people who paste a password with a trailing space, people

who cannot see your red border.

It also matters because the honest version of this knowledge is uncomfortable. Client-side validation is not security. Anyone actually attacking your application skips the form entirely and sends requests straight to the server. Students who do not understand that build a beautiful form and ship a defenceless endpoint.

The scenario

Tunde builds a signup form. It validates well. A month after launch he has four hundred accounts, of which about ninety are real. The rest have names like "a" and "test", because an automated tool filled the form and nothing checked the shape of what arrived.

Separately, a real customer, Hauwa, cannot complete signup at all. She uses a screen reader. The submit button is disabled until the form is valid, and when it is disabled it leaves the tab order with no explanation, so her screen reader simply stops announcing anything. She assumes the site is broken and leaves.

Those two failures came from the same cause: the form was built for a cooperative sighted user with a mouse. That is what steps three and five of this task exist to prevent.

Steps

1. Collapse the authentication pages

Your agent will have generated separate pages for signup, login and password reset.

Collapse them into one page using conditional rendering, with the active form held in a URL query parameter.

Then prove it:

1. Record the page count before and after. Ask the agent to list the pages in the application and submit both answers.
2. Capture the address bar on each form state, showing the same route with a different parameter.
3. Capture the server log while switching between forms, showing that no new session is created and no page reload occurs.

Write docs/spa-rationale.md: why instant form switching matters, what the user experience was before, and one case where separate pages would be the better choice. That last part is required. Single page architecture is not universally correct and a student who cannot name its cost has learned a rule rather than a trade-off.

2. Information design on the form

Before any validation, decide what the form asks for.

List every field. For each one: why it is needed, what the business does with it, and what happens if the user gets it wrong. Any field that fails all three comes out.

Then two required items:

Terms and conditions. Add acceptance. You are collecting personal data and in most of these businesses taking money, so this is not optional. Write in docs/terms-note.md where your wording came from. Adapting an existing template or getting it checked is the honest answer. Writing your own legal text from scratch is not, and claiming an agent wrote your terms and that is fine is a wrong answer you should be ready to defend against.

Remove the decoration that is not functional. In class the logo inside the form was removed because clicking it did nothing. Find your equivalent.

3. Client-side validation, in four categories

Build all four. Capture each one working.

Category one: empty fields. An inline error appearing when a user focuses a field and leaves without typing. Your wording, not the wording we used in class.

Category two: length and structure. For every field that has a shape, enforce it.

For a name field, we enforced a minimum of two characters and at least two words separated by a space, because bots and agents fill name fields with single tokens and because collecting a real name is a compliance matter.

Your business has different fields. Derive each rule and record the derivation:

- What the rule is.
- What goes wrong without it.
- What legitimate user it will wrongly block, because every rule blocks someone.

That third point is required for every rule. A two-word name rule blocks people who genuinely have one name, and there are many. You either accept that cost knowingly or you design around it. Pretending the cost does not exist is the failure.

Category three: format, in real time. Email format is the obvious one. The valid shape is a string, then an at character, then a domain.

Two things you must get right and defend:

Real-time enforcement belongs on registration, not on login. On registration you are collecting data for the first time and some friction is appropriate, like a bank being strict when you open an account. On login, checking format aggressively as someone types is just irritating, and you enforce it on submit instead. Implement both behaviours and capture both.

Applications do not check whether an email exists. You check the format. Ownership is proved later by sending a code. Write two sentences on why checking existence would be pointless even if it were possible.

Category four: ARIA labels and accessible errors. Covered in step five.

4. One field where the obvious input type is the wrong one

We used a text input restricted to numbers rather than a number input for the member number.

The reason is user experience. A number input shows a default zero and spinners. Correcting an entry becomes: clear 400, get 0, type 400, get 0400, go back, delete the zero. The fix is a text input that accepts only digits, converted with the number constructor in one line before it reaches the server.

Find the equivalent field in your business. Implement the text input version. Then:

1. Capture the number input being annoying in exactly that way. A short screen recording of thirty seconds is ideal. A sequence of three screenshots showing the zero, the 0400 and the cursor going back to delete it is accepted in full.
2. Record the same field as a text input, behaving properly.
3. Write two sentences on where the number input would be the correct choice.

5. Do not leak the shape of sensitive data

We removed a placeholder that revealed the member number was six digits, on the grounds that the number functions like a password and you do not give clues about special data. It is the same reason a failed login says "incorrect details" rather than naming which half was wrong.

In your business, identify at least one field where the length or format is itself sensitive. Remove the hint. Then write docs/enumeration.md:

- Which field, and why its shape is sensitive.
- What an attacker gains from knowing the shape.
- The trade-off you accepted, because removing the hint makes the form harder for legitimate users. Say how much harder and whether you think it is worth it.

6. The accessibility audit

Red borders alone fail. Users in the red and green colour blindness spectrum do not see the red, so the anxiety the colour is meant to create never reaches them. They see text with no signal attached.

Build, then audit, then fix:

1. Add a caution icon beside every inline error, alongside the colour.
2. Run an audit of your form. Not a tool run. A written audit by you, field by field, against: colour independence, keyboard reachability, focus visibility, screen reader announcement, and error association.
3. Find at least four real problems. We found exactly four in our own form: the disabled button leaving the tab order with no programmatic explanation, the acceptance signal being mouse only, the privacy checkbox with no aria-required, and the password requirements not linked with aria-describedby.
4. Fix all four.
5. Test with an actual screen reader. Write down exactly what it announced, field by field, before your fixes and after them, in docs/accessibility-audit.md. The written transcript is the requirement and it is accepted in full. Thirty seconds of audio of the form being read aloud, before and after, earns bonus marks on top.

Running the screen reader is not optional. Recording it is. What cannot be skipped is the transcript, because the exact words a screen reader says about your form are not something you can guess or generate. If you have never set one up, your operating system already has one built in and it takes about ten minutes to learn.

7. Password rules, and the button

Password. Length beats complexity. Set your minimum and justify the number. Show the hint only once typing begins rather than having it sit there by default. Offer either a confirm field or a visibility toggle, and say which you chose and why the other would be better in a stricter industry.

The button. Show it by default, disabled until the form is valid.

We tried hiding it entirely until the form was complete and rejected that, because users expect to see the button. Implement the hidden version first, capture it, then implement the disabled version, and write two sentences on why the second is better even though the first looks tidier.

Add a message that fires when someone clicks the disabled button without accepting terms, so the disabled state explains itself rather than just refusing.

8. Attack your own form, then skip it entirely

Two parts, and the second is the important one.

Through the form. Try to defeat every rule you wrote. Single-character names, a password of spaces, an email of a@b, a field pasted with trailing whitespace, every field at maximum length. Record what got through. Something will.

Around the form. Send a request straight to your endpoint, bypassing the browser entirely. Use whatever command-line tool you have.

Submit the exact command and the exact response, as text.

Then write docs/client-side-is-not-security.md, which is the most important document in this task:

- The request you sent and what your server did with it.
- Which of your eight client-side rules were enforced on that request. Most likely none.
- What this proves about the relationship between client validation and security.
- What you will do about it in task four.

If your server accepted that request, say so plainly. That is the correct and expected result at this stage, and hiding it is worse than having it.

9. Roll back without hallucination

Make a change you regret. Then roll it back properly.

Not by asking the agent to undo the last two versions, which invites it to guess. List your commits, identify the specific commit ID, and roll back to that ID.

Submit the commit list, the ID you chose, the command you ran, and two sentences on why naming the ID is safer than describing the change in words.

Deliverables

task-03-client-validation/ README.md docs/ spa-rationale.md field-justification.md every field, three questions each validation-rules.md each rule, what breaks without it, who it wrongly blocks terms-note.md enumeration.md accessibility-audit.md field by field, five criteria client-side-is-not-security.md rollback.md defects.md interview-answers.md evidence/ 01-page-count-before.txt 02-page-count-after.txt 03-url-states.png 04-form-switch-logs.txt 05-empty-field-error.png 06-length-rule-blocking.png 07-email-realtime-signup.png 08-email-on-submit-login.png 09-number-input-annoying.mp4 or three screenshots 10-text-input-working.mp4 or three screenshots 11-screenreader-before.mp3 optional, bonus marks 12-screenreader-after.mp3 optional, bonus marks 13-button-hidden-version.png 14-button-disabled-version.png 15-form-attack-results.txt 16-direct-request-command-and-response.txt 17-commit-list-and-rollback.txt 18-walkthrough.mp4 optional, bonus marks

Optional walkthrough, up to five bonus marks. Five minutes. Attack your own form, then send the direct request to your endpoint and read out what comes back.

The interview defence

Answer all fourteen in docs/interview-answers.md. Two hundred words each.

Validation. Questions one to five are published interview questions, taken verbatim or near-verbatim.

1. Why is client-side validation not enough?
2. What security checks belong in the frontend, and what must the server enforce regardless?
3. The user interface hides the delete button for non-administrators. Is that enough?
4. What is the difference between validation, encoding and sanitisation?
5. Why should you validate route parameters and query strings, not just the body?

Architecture. Published questions.

6. Explain what a single page application is, and how you make one discoverable by search engines.
7. What are the advantages and disadvantages of a single page application against a multi-page application?

Accessibility. Published questions.

8. Can you explain the four principles of the Web Content Accessibility Guidelines, with examples from your own work?
9. Discuss the importance of semantic HTML in the context of accessibility.

The engagement. Published forward deployed engineer questions.

10. Tell me about a time when you disagreed with what a customer wanted to build.

11. How would you explain to a non-technical executive why their AI implementation is not performing as expected?

Then three about your work, written for this assessment:

12. One of your validation rules wrongly blocks a category of legitimate user. Name the rule, name the users, and defend the trade-off or say you got it wrong.
13. You sent a request directly to your own endpoint. Say exactly what happened and what it tells you about the eight rules you spent two days writing.
14. **Wrong-answer analysis.** A candidate says: "I use strong client-side validation with regex on every field, disable the submit button until everything is valid, and sanitise all inputs in the browser before sending them. That covers the main injection and bad-data risks." This is confident and substantially wrong. Say which parts are useless, which are actively misleading, and what the candidate has confused with what.

One more, which has no published interview question anywhere and which you should be able to answer anyway: what is user enumeration, and how do error messages contribute to it? Published sets ask about information disclosure in error messages generally. They do not ask about account enumeration specifically, which is a gap in the question sets rather than in the subject.

How this is graded

- The four validation categories, with derived rather than copied rules: 20
- The accessibility audit and the screen reader transcripts: 25
- The direct-request investigation and its write-up: 25
- Rules documented with the users they wrongly block: 15
- Viva: 15
- Optional walkthrough: up to 5 bonus marks on top of the 100

Automatic fail

- No screen reader transcript, before and after.
- Validation rules copied from class rather than derived from your business.
- No direct request to the endpoint, or no output from it.
- A rule with no stated cost to legitimate users.

TASK FOUR: THE SERVER IS THE SOURCE OF TRUTH

What this draws on: schema design, migrations, email delivery, server-side validation, hashing, rate limiting and email verification.

Window: eight days. This is the largest task.

What this task tests

Whether you can design a schema, run migrations under control, send email without blocking the request, and build server-side defences that hold when someone ignores your form completely.

Why it matters

Task three ended with you sending a request straight to your own endpoint and your server probably accepting it. This task closes that.

The server is the source of truth. Backend protection exists to stop tools forcing unvalidated data into your database, which compromises the entire application rather than one form.

This is also the block that gets interviewed hardest. Password hashing, session handling, rate limiting and validation placement appear in screening rounds for nearly every engineering role, agentic or not, and they have standard wrong answers common enough that interviewers test for them specifically.

The scenario

Chijioke runs a driving school with about three hundred students. His platform was built by a contractor who left. One morning he gets an email saying his student list is for sale on a forum.

What happened: the passwords were hashed with SHA-256, which is fast. The attacker copied the table and recovered most of the passwords within a day, because people reuse passwords and a fast algorithm is exactly what a brute-force attack wants. Three of those students used the same password on their bank.

Nothing was unpatched. No zero-day was involved. One algorithm choice, made once, by someone who thought fast was good.

That is why step four of this task is about why slow is correct, and it is the single most reliable interview question in this whole document.

Steps

1. Design the schema before you write it

On paper first, photographed, then in code.

Your schema must include at least five tables and must include all four of these:

1. The people who use the system.
2. Their sessions.
3. A money table, and separately a money-attempts table.
4. The chunks for your retrieval system, whether or not you build retrieval yet.

For each table, in docs/schema.md:

· Every column, its type, and whether it can be empty.

- Every relationship, marked one to many or many to many.
- Every index, with one line on why that column and what it costs on writes.
- Where the same fact is stored twice, if anywhere, and why you accepted that.

The money-attempts table. Separate from payments, and most people do not build it. It records attempts that failed, were abandoned, or died on a bad network.

Two reasons, and you need both in writing. First, drop-off data tells the business where it loses people. Second, and more important, payment logs are the source of truth for litigation, government inquiries and annual tax filing. Write one paragraph on what you would do if your tax authority asked you to account for a specific transaction from fourteen months ago.

2. Discover what state your database is actually in

Open your database inspector before you run anything.

In class this revealed an empty database. A schema file existed. No tables and no migrations had ever been run. The application had been built for days against nothing.

Capture what you find, whatever it is. If your database is already correct, capture that and say how you verified it.

3. Migrations, run deliberately

A migration is any instruction given to your ORM to change data, structure or storage.

Four things to do and evidence:

Run the initial migration. Capture the command and the full output.

Add a required column to a populated table. It will fail, because existing rows have no value for it. Capture the failure. Then do it in three steps: add it as optional, fill it, make it required. Capture all three.

This is the step that separates people who have run migrations from people who have watched an agent run them.

Hit the drift error on purpose. Change your schema file without migrating, then run the application. You will get a client error, because generating the client rebuilds the types without touching the database. Capture the error, explain in one paragraph what the two things are that have gone out of step, and then fix it.

Keep the agent out of it. Your rules should prohibit the agent from running migrations. In class the agent correctly did not run one, and the migration had to be run by hand, which is the propose-and-stop rule working as designed.

Write docs/migrations.md: why forward-only, why you never edit a migration that has already run, and why you would rather run these by hand than let an agent do it. Then state the cost of that choice honestly, because running them by hand is slower and people forget.

4. Hashing, and why fast is wrong

Choose your algorithm. Argon2id or bcrypt. Not SHA-256, SHA-512 or MD5.

Then prove you understand it rather than reciting it:

1. Hash the same password twice. Submit both outputs. They differ, and you explain why in one line.
2. Time your chosen algorithm hashing one password. Then time SHA-256 hashing the same password. Submit both numbers from your own machine.
3. Using your own two numbers, calculate how many guesses per second an attacker gets against each. Show the arithmetic.
4. Write the inversion in your own words: you hash once per login, an attacker hashes billions of times, so slowing it down costs you nothing measurable and costs them everything.

Step three is the one that cannot be faked, because the numbers are from your hardware.

Also record: what salting does, whether your algorithm handles it for you, and what an attacker could do if you skipped it.

5. Normalisation and trimming

Email, lowercased, on every form that takes an email: registration, login, reset. A user who typed a capital letter on their phone must still be able to log in from a laptop.

Passwords, never normalised, so users can remember the exact characters they chose.

Passwords, trimmed of leading and trailing whitespace, because copying a password commonly picks up a space and the login then fails for a reason the user cannot see or diagnose.

Prove all three. Register with a capital in the email, log in with it lowercase, and capture both. Then register with a password containing a trailing space, log in without it, and capture that.

Write two sentences on why you trim passwords but do not lowercase them, since both are changes to what the user typed.

6. Schema validation at the boundary

Install a validation library. Zod, or another, with your reason recorded.

It sits at the backend and checks every incoming request against a defined shape before anything else touches it.

Apply it to registration, login and reset.

Then the measurement most people skip: time a request before and after. In class, validation added a noticeable two to three second delay on signup. Find your number and submit it. If your number is high, say what you think is causing it rather than accepting it.

Write docs/validation-placement.md: where in the request lifecycle validation sits, what runs before it, what must never run before it, and why validating in your route handler by hand is worse than a schema even though it looks simpler.

7. Every client-side rule, enforced again on the server

Take the rules from task three. All of them. Enforce each on the server.

Then repeat your task three attack. Send the same direct request that got through before. Capture the refusal.

Submit a side-by-side file: the request, the task three response, and the task four response. This is the clearest single piece of evidence in the whole assessment that you understood the lesson.

8. Rate limiting, and its limits

Implement it on registration, login and reset. Choose your threshold and window and justify both numbers.

Then:

1. Trigger it. Capture the lockout message and the status code.
2. Defeat it. Change your IP address, by any legitimate means available to you, and get past your own limit. Capture it.
3. Write docs/rate-limiting.md: why IP alone is insufficient, what else serious applications key on, including the email address and hardware identifiers, where your counters are stored and the trade-off of that choice, and what your limit does to a legitimate user on a shared office or campus connection.

Point two is the part nobody does, and it is the part that proves you understand the control rather than having installed it.

9. Breached password checks

Users reuse passwords that have already leaked, and attackers try leaked passwords against every application they find.

Implement a check against a breached password service, on registration only.

Then explain the mechanism, which is the actual interview question: you do not send the password. You send a short prefix of its hash and receive back a range of matches, then check locally. Write how that works in your own words, in docs/breach-check.md, and say why the naive version would be an unacceptable thing to build.

Test it with a password you know is in the corpus. Capture the rejection.

10. Email verification

Generation. A six-digit code, generated with your platform's cryptographic random function. Not the ordinary pseudorandom one, which is predictable and not built for this.

Write one paragraph on what an attacker could do against predictable codes, given your expiry window and your rate limit.

Expiry. We set two minutes, then decided it was too short, because a real person has to find their phone and open their email. Five, seven or ten minutes is more realistic.

Pick your number. Justify it against two things: how long your users actually need, and what the window costs you in attack surface.

Delivery. Configure SMTP. Record your host and port. Expect the first attempt to fail. Ours failed because Google rejected the login: the configuration held an ordinary password rather than an application

password. Capture that failure and the fix.

Background. The verification email, and any welcome email, goes to a background job. Not inside the registration request.

The reason is performance and bandwidth. Sending inside the request slows the application when several people register at once, and ties your registration to someone else's mail server being up. Users commonly receive a welcome email after they have already logged out, and that is correct behaviour.

Prove it: capture the registration response time with the email inside the request, and again with it in a job. Two numbers from your machine.

The flow, tested. Register. Retrieve the code from your database, capture the row. Enter it. Capture the verified state. Then capture three failures: an expired code, a wrong code, and the same valid code used twice.

That third one is a real defect we found in our own build, where verification was not retry-safe after a successful verification. If yours has it, document it rather than quietly fixing it.

11. The email has to look like your product

The agent will design the email. In class it came out a shade of purple instead of the brand blue, because one model used the design system and another did not.

Make yours use your task one tokens. Capture the rendered email. If it comes out wrong, capture the wrong version too and say what you traced it to.

Then one honest note in docs/email-copy.md: the agent wrote your copy, and in a real business a developer or marketer writes it, because the copy is the marketing. Say what you would change about the wording if this were going to real customers.

12. What you decided not to do

List at least four security controls you considered and did not implement. For each: what it is, why it was not worth it for your business at this stage, and what would have to change for you to add it.

This section is weighted. Knowing what you left out is the difference between a judgement and a checklist.

Deliverables

task-04-server-security/ README.md docs/ schema.md migrations.md hashing.md with your two timing numbers and the arithmetic validation-placement.md rate-limiting.md breach-check.md verification-design.md code generation, expiry justification, attack note email-copy.md not-implemented.md the four controls you skipped defects.md interview-answers.md evidence/ 01-schema-drawing.jpg 02-database-state-before.png 03-initial-migration.txt 04-required-column-failure.txt 05-required-column-three-steps.txt 06-drift-error.txt 07-hash-twice-different.txt 08-timing-argon-vs-sha.txt 09-email-case-normalised.png 10-password-trim-working.png 11-validation-timing.txt 12-direct-request-before-and-after.txt 13-rate-limit-triggered.png 14-rate-limit-defeated.txt 15-breach-check-rejection.png 16-smtp-failure.txt

17-smtp-success.txt 18-registration-timing-sync-vs-job.txt 19-verification-row.png 20-verification-expired.png 21-verification-wrong-code.png 22-verification-code-reused.png 23-rendered-email.png 24-walkthrough.mp4 optional, bonus marks

Optional walkthrough, up to five bonus marks. Seven minutes. Run your hashing timing comparison. Trigger your rate limit. Then send the direct request from task three and show it being refused. This is the one most worth recording, because these three things are what you will be asked to describe in an interview.

The interview defence

Answer all nineteen in docs/interview-answers.md. Two hundred words each. This is the most heavily interviewed material in the whole document.

Hashing. Questions one to five are published interview questions, taken verbatim or near-verbatim.

1. Why should you hash passwords?
2. Why use bcrypt instead of SHA-256 for passwords?
3. What is a salt and why do you need it?
4. What salt rounds, or what work factor, should you use? Give a number and defend it.
5. You are storing user passwords in a database. Of plain text, SHA-256, AES-256 encryption and adaptive hashing, which is correct and why are the other three wrong? Note that one of the three wrong answers is wrong for a different reason from the other two.

Validation. Published questions.

6. Why is client-side validation not enough?
7. What security checks belong in the frontend, and what must the server enforce regardless?
8. The user interface hides the delete button for non-administrators. Is that enough? If not, what does an attacker do instead?
9. What is the difference between validation, encoding and sanitisation?
10. Why would you replace the incoming request body with the parsed and validated data rather than leaving both available?
11. Why validate route parameters and query strings, not just the body?

Rate limiting. Published questions.

12. How do you rate limit unauthenticated requests, and what is the problem with the header you would reach for first?
13. Which rate limiting algorithm would you choose for a production API, and what is the boundary burst problem?
14. Why use a shared store for your counters rather than in-memory counters on each server?
15. What HTTP status code should a rate limiter return, and why is 403 the wrong choice?
16. What do you do when your rate limit store is unavailable: fail open or fail closed? Answer for your own application specifically.

Then three about your work, written for this assessment.

17. You have two timing numbers for hashing, from your own machine. Walk me through what they mean for an attacker who has stolen your user table.
18. How does a breached password check avoid sending the password anywhere? Explain the mechanism in your own words, then say what the naive version would leak.
19. **Wrong-answer analysis.** A candidate says: "I hash passwords with SHA-256 plus a random salt per user, which means even if the database is stolen the passwords cannot be reversed, because hashing is one-way. I also rate limit login attempts by IP, so brute force is not possible." Every sentence contains something true. Say what is wrong, in order of severity, and what the candidate has misunderstood about the threat model.

Also prepare these for the viva. No written answers needed. The published ones come first. The last three appear in no published question set at all, which is worth knowing: migrations get interviewed through the zero-downtime question rather than through the mechanics.

Databases, published:

- A query that was running in ten milliseconds suddenly takes thirty seconds. Walk me through your debugging process.
- How would you index a table to optimise both a query filtering on one column and a query filtering on that column plus a second one?
- Why does the order of columns in a composite index matter?
- Why not create an index for every frequently filtered column?
- What is the N plus one problem, and how do you solve it?
- Can you explain how you would migrate a large database with minimal downtime?

Queues and email, published:

- What is the difference between at-least-once and exactly-once task execution?
- How do you make a queue job handler idempotent?
- A queue is backing up and jobs are being enqueued faster than workers can process them. What do you do?
- What is a dead-letter queue, and how do you use it in production?
- What is the problem with sending an email synchronously inside a registration flow, and how would you redesign it?

Not asked anywhere, and you should still be able to answer:

- What is a forward-only migration, and why would you never edit one that has already run?
- What happens when a migration fails halfway through?
- What is the difference between generating your ORM client and running a migration?

How this is graded

- Schema design, including the money-attempts table and its reasoning: 15

- Migrations, including the failed required-column attempt: 15
- Hashing understanding, proved with your own timing numbers: 15
- Rate limiting, including defeating your own limit: 15
- The before and after on the direct request: 10
- Email verification flow, including the three failure cases: 10
- The four controls you chose not to implement: 5
- Viva: 15
- Optional walkthrough: up to 5 bonus marks on top of the 100

Automatic fail

- SHA-256, SHA-512 or MD5 used for passwords.
- Timing numbers not from your own machine, or no arithmetic shown.
- Rate limiting implemented but never defeated.
- No captured migration failure.
- Fewer than four controls in the not-implemented list.

TASK FIVE: SESSIONS, DELETION, AND THE DEFENCE

What this draws on: sessions, middleware, logout, account deletion and compliance, plus everything in tasks one to four.

Window: nine days. The written work is the largest part of this task and it should be started on day one, not day seven.

What this task tests

Three things. Whether you can hold a session securely and end it properly. Whether you can delete a user's account without breaking the law. And whether you can explain any of the work from the last several weeks to a person who is deciding whether to hire you.

Why it matters

A session is created the moment a user logs in. Without tracking, sessions can be hijacked, which is the same shape of problem as someone stealing an office ID badge: the attacker then acts as the account owner, and the logs say the owner did it.

The deletion half is where this stops being an engineering question. It is illegal for an application owner to delete a user's account by hand from the database. Under GDPR in Europe or the NDPA in Nigeria, a deletion request goes through a process, not a wipe. Most people building these applications do not know that, and it is exactly the kind of thing a client's lawyer asks about before signing.

And the defence matters because everything in tasks one to four is worth nothing in a hiring process you cannot talk your way through. Employers do not hire claims. They hire evidence, and then they ask you about it for forty-five minutes.

The scenario

Hauwa runs a cooperative society with four hundred members and about nine million naira moving through it each month. A member, Ibrahim, leaves and asks for his account to be deleted. Hauwa opens the database and deletes his row.

Three things happen.

Ibrahim's two years of contribution records go with him, so the cooperative's books no longer balance and the discrepancy is unexplainable. Six weeks later Ibrahim disputes a payout and there is no record either way. And when the cooperative is audited, the auditor finds a gap and has to treat it as a possible concealment rather than a deletion request.

Hauwa did what Ibrahim asked. She did it in the one way that was both illegal and self-harming. Steps five to eight of this task are what she should have built instead.

Steps

1. Sessions, built and understood

Implement session handling. On login, the server generates a unique random session identifier using your platform's cryptographic random function and sends it to the browser.

Requirements, each of which you evidence:

1. A fresh token on every sign-in. Never a reused one. Log in twice and capture two different identifiers.
2. The session record stored server side, in your sessions table.
3. The session cookie hashed.
4. Cookie flags set correctly. Record which flags you set and what each one prevents.

Then write docs/sessions.md: what session hijacking is, how a stolen identifier is used, what logging out does to it, and what your application does that limits the damage if one is stolen.

2. Soft gating or hard gating, decided

Soft gating. The new user is created and logged in immediately, with no verification first. Most applications do this.

Hard gating. The user is stopped after account creation, must verify by email, and must then log in manually. Used in finance, government technology, fintech and health technology.

Choose for your business. Then do the harder part: implement the one you did not choose, on a branch, and capture both flows.

Write docs/gating.md: which you shipped, why, what the other costs in completed registrations, and the specific regulatory or trust condition that would make you switch.

A student who has only built one of these has an opinion. A student who has built both has a judgement.

3. The login flow, to specification

Every item, implemented and evidenced:

1. Schema validation on the login payload.
2. Email normalised, trimmed and lowercased.
3. Database lookup by the identifying field, not by name.
4. Password verified with the same algorithm that hashed it.
5. Rate limiting applied.
6. Generic error messages. "Incorrect email or password", never naming which half failed.
7. A verification status check, with your chosen behaviour for unverified users.
8. A fresh session token issued.

For item six, prove it: attempt a login with a correct email and wrong password, then a wrong email entirely. Capture both responses. They must be indistinguishable. Then write two sentences on what an attacker learns if they are not.

For item seven, we chose to let unverified members log in with a warning banner rather than blocking them. State your choice and the consequence you accepted.

4. Middleware, and logout done properly

Middleware. It reads the session cookie and redirects anyone unauthenticated to the login page. It also checks the user is still valid and not banned, which is a compliance matter rather than a convenience. And it attaches the user's details to the request, so the browser holds the right record of who is logged in.

That last item has a specific failure: without it, two people sharing one browser get confused with each other. Test that. Log in as one user, log in as another in the same browser, and capture what your application does. Fix it if it is wrong.

Logout. Badly built applications log out on the front end only. The interface changes, the user is returned to the login form, and the session stays live in the database until it expires.

Yours deletes the session row on the server and clears the cookie.

Prove it: log in, capture the session row, log out, capture the session row gone.

The edge case. A network failure makes the logout request fail on the server while the interface shows the user as logged out. Their session stays active and they believe it does not.

Reproduce it. Throttle or block the request, capture the interface claiming success and the session row still present. Then fix it, and capture the fix.

This is a real defect we found in our own build, and it is the kind of thing an interviewer remembers you for.

5. Password reset kills everything

Ordinary sessions can persist. A password reset cannot.

On reset, every previous session and cookie is wiped and the user logs in again from scratch.

Prove it: log in on two browsers, reset the password from one, and capture the other browser being logged out.

Write two sentences on why this is different from an ordinary password change and which industries would treat both the same way.

6. Account deletion, as a legal process

The four terms, and you will be asked to define them:

Soft delete. Marking the account as scheduled for deletion rather than removing it.

Grace period. A window where the account is locked and inactive but still in the database. Facebook's model.

Hard delete. Purging the data once the grace period ends.

Scheduled job. A background task on a timer that finds expired accounts and deletes them.

Implement all four. Your grace period length is yours to choose and justify. We used seven days.

Requirements:

1. The account locks immediately on request, which revokes all active sessions.
2. Identity is anonymised.
3. A scheduled job runs on your chosen interval, finds expired requests, and purges in a single transaction.
4. The job logs a start line and a finish line with counts. A job that stopped running looks exactly like a job with nothing to do, and the counts are what tell them apart.

Prove the job works and prove it is safe to run twice. Run it twice against the same data and capture that the second run changes nothing.

7. The retention matrix

The document that matters most in this task. docs/retention-matrix.md.

Every table and every significant column in your schema, in one list. For each:

- Purged or retained.
- The reason.
- The legal or business basis for retaining it, if retained.
- How long, if retained for a fixed period.

The line we have drawn so far in Spotter is this. Purged in a single transaction: sessions, check-in codes, attendance, question logs including wrong-answer reports, access logs. Retained permanently: payment

records, ledger entries, payment attempts, membership changes. The position we took was that financial data is always retained, down to two kobo, while AI exercise questions, phone numbers and email addresses can go.

Your business is not a gym, so your matrix is not that matrix. Derive it.

Then three harder items:

Backups. Backups must sync with the live database to drop deleted rows, or the retention is itself illegal unless a government mandate or your terms permit it. State your sync interval and where your backups live.

The conflict. You will find at least one case where the right to erasure and a retention obligation point in opposite directions. Name it. Say which wins and on what basis.

Anonymisation against deletion. Say where you anonymise rather than delete, and why anonymised data can be kept when identifiable data cannot.

8. The privacy document, and the promise you had to remove

Write privacy.md in your repository. Plain language, not borrowed legal text you cannot explain.

It must state your grace period, what is purged, what is retained and why, and how a user requests deletion.

Then one specific exercise. In class the privacy document had to be edited to remove a line telling users they could request deletion at the front desk, because the application could not actually honour a request made that way.

Find your equivalent: one promise in your draft that your system cannot keep. Remove it. Record both versions and write two sentences on how a promise like that gets written in the first place.

9. Front-end state management, and whether you needed it

State management is the real-time handling of page and form data as it changes.

First, find out what you have. Ask your agent which state management library the application uses. In class the answer was none: it relied on local component state with the server as the source of truth.

Then implement one, with a real use for it. The class example: the forgot-password form captures and carries the user's email locally, so a failed login populates the reset form without the user retyping anything, with no server request and no backend cost.

Capture it working.

Then the honest question, in docs/state-management.md: did you need a library for this, or did you add one because it is expected? Name the point at which your application would genuinely require it. Many applications never reach that point and ship a library anyway.

10. The protected route and the second kind of user

Your application has one kind of user. Real businesses have at least two.

Build an administrative view behind a protected route. In Spotter this is the receptionist dashboard still ahead of us, where the secretary generates temporary passwords and member numbers that are emailed to members automatically. You are building yours before we build ours.

Requirements:

1. A separate role, stored on the user record.
2. A protected route that checks the role, not just whether someone is logged in.
3. One action only that role can perform.
4. The copied-link test: log in as an ordinary user, paste the administrative URL, and capture what happens. If you see the page, your route checks authentication and not authorisation, and you have found the most common serious hole in applications built this way.

Capture the attempt and the refusal.

11. Primary research: talk to a real person

One conversation with one real person who runs a business of the kind you are building for. Not a persona. A person with a name and a business you could walk into.

Twenty minutes. Ask them:

1. What do customers ask you repeatedly that only you can answer?
2. What records do you keep, and where do they live now?
3. Has a customer ever asked you to delete their information, and what did you do?
4. What would you do if your records were lost tomorrow?
5. What would make you not use software like this?

Submit docs/interview-with-owner.md: who they are, their business, the date, and what they actually said, including the answers that contradict your design.

Then one paragraph: what you got wrong about your own business brief from task one, now that you have spoken to someone in it.

If every answer confirmed your design, you asked leading questions. Say so.

12. The documentation of everything

The main written deliverable. docs/handbook.md.

A technical handbook covering all seven topic areas below, written for someone one cohort behind you. Not notes. An explanation.

Seven sections, one per session date, and each section must contain:

- **The concept, in plain words.** Define it before using it.
- **Why it exists.** The failure it prevents. Never state a rule without the reason.
- **A real scenario.** Named people, a specific problem, how it played out. Your own scenario, not the one we used in class.

- **How it is done.** The actual mechanism, specific enough to follow.
- **Where it goes wrong.** The failure mode, and what it costs.
- **What you got wrong.** Your own mistake on that topic, during these tasks.

The sections:

1. Typography, shadows, and the design system export.
2. Scaffolding, the development server, and database diagnostics.
3. Information design, the browser head, and responsive type.
4. The working tree, branching, and client-side validation.
5. Migrations, schema design, and email delivery.
6. Server-side validation, hashing, rate limiting and verification.
7. Sessions, logout, account deletion and compliance.

Length is whatever it takes. Padding is visible and is marked down. So is a section that defines a term without a scenario.

The last bullet in every section is the one that cannot be generated and is weighted accordingly. Seven real mistakes, specific enough that someone could verify them against your commit history.

13. Publish it

One written piece, published publicly. Substack, Medium, LinkedIn or your own site.

Not a summary of the course. One decision you made during these tasks, told as a story: the problem, what you tried, what went wrong, what you changed, what happened. With your prompts shown as evidence and at least one thing you got wrong.

Fifteen hundred words is plenty. Submit the live URL in your readme.

The test: a hiring manager who reads only this piece should be able to tell whether you understand the thing you are describing.

14. What you do not know

docs/what-i-do-not-know.md.

At least eight items. For each: what it is, how you know you do not understand it, and what you would need to learn it.

Honest examples from this body of work: how your session cookie is actually encrypted, what your ORM does when a migration fails halfway, whether your rate limit storage survives a restart, what happens to your scheduled job if the server is down when it should fire, whether your retention matrix would survive a lawyer reading it.

This document is read in the viva and questioned. A short or vague list is marked down, because the inability to locate your own ignorance is itself a finding.

Deliverables

task-05-sessions-compliance-defence/ README.md privacy.md privacy-draft-with-broken-promise.md docs/ sessions.md gating.md retention-matrix.md state-management.md interview-with-owner.md handbook.md seven sections what-i-do-not-know.md at least eight items defects.md interview-answers.md evidence/ 01-two-session-ids.txt 02-cookie-flags.png 03-soft-gating-flow.mp4 or a screenshot sequence 04-hard-gating-flow.mp4 or a screenshot sequence 05-generic-errors-both-cases.png 06-two-users-one-browser.txt 07-session-row-present.png 08-session-row-deleted.png 09-logout-network-failure.txt 10-logout-edge-case-fixed.txt 11-reset-kills-other-session.mp4 or two screenshots, both browsers 12-deletion-request-locks-account.png 13-scheduled-job-run-one.txt 14-scheduled-job-run-two.txt 15-state-management-working.mp4 or a screenshot sequence 16-admin-route-refused.png 17-published-article-screenshot.png 18-walkthrough.mp4 optional, bonus marks

Optional walkthrough, up to five bonus marks. Ten minutes, and the longest of the five. Walk through your retention matrix and defend three decisions in it. Then run your deletion job and show the second run changing nothing.

The interview defence

Answer all twenty-two in docs/interview-answers.md. Two hundred words each.

On sessions. Questions one to seven are published interview questions.

1. What should a secure browser session look like? Name every property and what each one prevents.
2. What is session fixation, and what prevents it?
3. Are tokens better than server-side sessions? Answer for your own application, not in general.
4. How would you implement log out of all devices?
5. How do you invalidate existing sessions on a password change?
6. What do the HttpOnly, Secure and SameSite cookie flags protect against, and what does each one fail to protect against?
7. What is the difference between a 401 and a 403, and how should the interface treat them differently?

On your own session work, written for this assessment:

8. Why must logout be handled on the server rather than the front end, and what exactly stays alive if it is not?
9. What is the difference between authentication and authorisation, and which one does a protected route need?

On compliance. Written for this assessment, because published interview sets do not cover this ground. You will be asked these by clients rather than by interviewers, and the stakes are higher.

10. A user asks you to delete their account. Walk me through what happens, from the request to the data being gone.
11. Which data would you refuse to delete on a deletion request, and on what basis?
12. What happens to your backups when a user is deleted?
13. What is the difference between anonymising and deleting, and when is anonymising sufficient?

On the role. Published questions from forward deployed engineer and AI engineer question sets.

14. Why do you want to work as a forward deployed engineer specifically, rather than as a software engineer?
15. A customer says they want to use AI to improve their operations. How do you respond?
16. What is the difference between a RAG system and a fine-tuned model for an enterprise deployment?
17. How do you evaluate whether an AI agent is working correctly in production?

On state management. Published questions.

18. How do you decide between local component state, context, and an external state manager?
19. How do you handle server state, as distinct from client state?

On your own work, written for this assessment, with cross-task consistency required:

20. Cite three specific numbers from your own tasks one to four: one from your type scale, one timing figure from your hashing comparison, and one from your rate limit configuration. Explain what each number means and why you chose it. Numbers that do not match your earlier submissions are a fail on this task.
21. Across all five tasks, name the decision you are least confident about. Say what you would need to find out to settle it, and what you would do if the answer went against you.
22. **Wrong-answer analysis.** A candidate says: "For GDPR compliance we added a delete account button that removes the user's row from the database immediately, so we are fully compliant with the right to be forgotten. We also keep backups for a year in case anyone needs their data restored." Two sentences, several problems. Identify each, say which is a legal exposure rather than a technical one, and say what the correct design is.

Also prepare these for the viva, published and directly relevant if you build retrieval into your application.

- What is the difference between RAG and fine-tuning, and when would you use each?
- What is chunking, and how do you choose the right chunk size?
- Why can a RAG system still hallucinate, and how would you reduce it?
- How do you evaluate the quality of a RAG system?
- How would you tell whether a wrong answer is a retrieval failure or a generation failure, using only the logs?
- When does a dedicated vector store make sense against running pgvector inside Postgres?
- Should structured records go into vector search?
- How would you handle access control in a RAG system?
- How would you keep a knowledge base up to date when source documents change?

How this is graded

- Sessions, middleware and the logout edge case: 15
- Deletion implemented as a process, with the job proved safe to run twice: 15

- The retention matrix, including the conflict you identified: 15
- The handbook, with seven real mistakes of your own: 20
- Primary research, including what it contradicted: 10
- The published piece: 5
- What you do not know: 5
- Viva: 15
- Optional walkthrough: up to 5 bonus marks on top of the 100

Automatic fail

- A retention matrix with no conflict identified.
- A handbook section with no scenario, or reusing our class scenario rather than writing your own.
- Primary research with no contradicting answer.
- Fewer than eight items in what you do not know.
- Numbers in question sixteen that contradict your earlier tasks.
- An administrative route reachable by a logged-in ordinary user.

THE VIVA PROTOCOL

Every task is defended out loud, in a live session with an assessor. This section explains how, because knowing the format in advance is fair and knowing the questions is not.

The viva is not the same thing as the optional walkthrough recording. The recording is yours, made in your own time, and you can skip it. The viva is live, it is scheduled, and it counts for fifteen percent of every task.

Format

Twenty minutes per task for tasks one to three. Thirty minutes for tasks four and five.

Three parts:

Part one, five minutes. You drive. Open the application, open the code, and walk through the thing you are proudest of. You choose what.

Part two, the bulk. The assessor drives. Questions written after reading your submission. They are about your decisions, your numbers and your defects, not about the topic in general. This is why preparation cannot save a submission you did not build.

Part three, five minutes. The one you did not expect. One request to change something, live. A validation rule to add, a value to adjust, a query to write. Small, and it has to work before the session ends.

What the assessor is told to look for

Four signals, in this order.

Can you find your own code. You should know roughly where things live without searching for a minute. A student who built it knows the shape of it.

Do your explanations match your code. The most common failure is a confident explanation of a mechanism the code does not actually implement. This is what agent-built-and-unread produces, and it is obvious within two questions.

Do you say when you do not know. Saying so, saying what you would check, and saying how you would find out scores better than a confident guess. Guessing when you do not know is the answer that loses roles people were otherwise winning.

Can you defend a decision against pressure. The assessor will push back on at least one choice, including choices that are correct. Changing your answer because someone frowned is a weaker signal than holding a defensible position.

What ends a viva early

- Not being able to find a file you wrote.
- Describing a mechanism your code does not contain.
- Terminal output you cannot reproduce live.
- A defect list that does not match anything in the code.

ACROSS ALL FIVE TASKS

The thread you are building

Each task feeds the next, deliberately.

Task one produces the design system. Task two consumes it while scaffolding and proves the agent used it. Task three builds the form inside that system and ends by showing you that none of its rules are security. Task four closes that hole and gives you the numbers you will cite later. Task five holds the session, deletes the account lawfully, and makes you explain all of it.

By the end you have one application, one repository, one commit history spanning five or six weeks, and a documentation set that covers every session. That is the artifact, and the artifact is the point.

Total weighting

- Task one, design system: 15
- Task two, repository and scaffolding: 20
- Task three, client-side and accessibility: 15
- Task four, server-side security: 30
- Task five, sessions, compliance and defence: 20

Task four carries the most because it is the most heavily interviewed material and the hardest to fake. Task five is second because the written work in it is what a hiring manager actually reads.

The documents that decide your grade

Most students will assume the working software decides it. It does not. Reasoning is thirty-five percent of every task.

Five documents carry disproportionate weight:

1. task-01/docs/divergences.md, because it asks where your own choice fails.
2. task-03/docs/client-side-is-not-security.md, because it asks you to report your own application being defeated.
3. task-04/docs/not-implemented.md, because it asks what you chose to leave out.
4. task-05/docs/retention-matrix.md, because it is the one document in this set that a lawyer would read.
5. task-05/docs/what-i-do-not-know.md, because locating your own ignorance is the skill the rest of the course depends on.

None of those five can be produced by a model that did not do the work, which is why they are weighted the way they are.

Advice

Start the writing on day one of each task. The decision log written as you go is a record. The same document written at the end is a reconstruction, and the difference is visible in three sentences.

Capture evidence as it happens. You cannot go back and photograph a port conflict you already fixed. Keep a terminal log open and save it as you work.

Make the mistakes on purpose where the task asks you to. Several steps require a broken state. Students who avoid breaking things lose those marks and learn less.

Commit constantly. In class the application reached forty-five uncommitted versions before anyone noticed. Twelve commits across three days is a floor, not a target.

Use the agent. This is not a test of whether you can code without one. It is a test of whether you can direct one and then tell whether it did what you asked. Those are different skills and the second one is the job.

WHERE THE QUESTIONS CAME FROM, AND WHAT IS NOT ASKED

Every question marked published in this document was taken from a page that lists it. Where a question has been shortened it has been shortened for length only, not changed in meaning. Questions about your own work, the wrong-answer analyses and the cross-task consistency question were written for this assessment.

One finding worth more than the questions

Putting these tasks together meant going through roughly two hundred published interview questions across every topic covered here. That also produced a list of things nobody asks, which is arguably more useful to you.

No published interview set asks about:

- Why AI-generated interfaces look generic, or what you hand an agent to prevent it.
- Keeping a design tool and a codebase as one source of truth.
- Semantic typography. The nearest published question is about semantic HTML.
- Forward-only migrations as a named concept, what happens when a migration fails halfway, or why you never edit a migration that has already run. Migrations are interviewed through the zero-downtime question instead.
- Anything specific to Prisma, including the difference between generating the client and running a migration.
- How a breached password check avoids sending the password.
- Using a cryptographic random function rather than an ordinary one for a verification code.
- Account enumeration specifically. Published sets ask about information disclosure in error messages generally.
- Cron jobs against event-triggered jobs.
- Why a background job must log a start line and a finish line.
- SMTP configuration.
- Argon2id against bcrypt head to head. Argon2id appears only inside answers to the bcrypt question.

That list is the useful part. Several of those are things you now do routinely and will not be asked about, which means you have to raise them yourself. An interviewer who never asks about your retention matrix will still be impressed that you have one, but only if you bring it up.

The reverse also holds. Password hashing, session security, rate limiting, git history rewriting, indexing and the N plus one problem are asked constantly, with standard wrong answers common enough that the questions exist specifically to catch them. Those are the ones to be word-perfect on.

Sources

Forward deployed engineer and AI engineer role questions:

- Forward Deployed Engineer Interview Questions: The 50 Questions Every Candidate Needs to Prepare For in 2026, https://techcareers.io/forward-deployed/forward-deployed-engineer-interview-questions-50-questions-2026
- Forward Deployed Engineer interview questions (2026): every round, with real examples, https://a10x.dev/blog/forward-deployed-engineer-interview-questions
- Forward Deployed Engineer Interview: The Definitive 2026 Guide, https://careerservices.fas.harvard.edu/blog/2026/08/01/forward-deployed-engineer-interview-the-definitive-2026-guide-fde/

Authentication, sessions and cookies:

- Authentication and Authorization Interview Questions: OAuth, OIDC, JWT and Sessions, https://prachub.com/resources/authentication-and-authorization-interview-questions-oauth-oidc-jwt-and-sessions
- Frontend Web Security Interview Questions, https://www.greatfrontend.com/blog/frontend-web-security-interview-questions
- Auth Interview Questions for Node.js, https://namastedev.com/topics/authentication-jwt-cookies/auth-interview-questions-nodejs
- HTTP Cookies Interview Questions, https://interviewprep.org/http-cookies-interview-questions/
- JWT Authentication Interview Questions, https://climbtheladder.com/jwt-authentication-interview-questions/
- Node.js Security Interview Questions, https://www.vskills.in/interview-questions/security-interview-questions/nodejs-security-interview-questions

Password hashing:

- Password Security Interview Questions, https://namastedev.com/topics/encrypting-passwords/password-security-interview-questions
- Password Storage Best Practices in Node.js, https://namastedev.com/topics/encrypting-passwords/password-storage-best-practices-in-nodejs
- OWASP Top 10 2025 Quiz: Test Your AppSec Interview Readiness, 51 Questions, https://dev.to/fosres/owasp-top-10-2025-quiz-test-your-appsec-interview-readiness-51-questions-397n

Validation:

- Why is client-side validation not enough?, https://namastedev.com/pages/faq/why-is-client-side-validation-not-enough
- Input Validation with Zod in Express, https://namastedev.com/topics/data-sanitization-schema-validations/input-validation-with-zod-in-express
- Express.js Developer Interview Guide, https://www.secondtalent.com/interview-guide/express-js-developer/

Rate limiting:

- System Design: Rate Limiter, https://www.techinterview.org/post/3233460467/system-design-rate-limiter/
- Design a Rate Limiter: Interview Walkthrough, https://www.techinterview.org/post/3233476911/design-rate-limiter-interview-walkthrough/
- API Security Interview Questions, https://www.techinterview.org/post/3233460404/api-security-interview-questions/

Git:

- Git and GitHub Interview Questions, https://lets-code.co.in/interview/git-github-interview-questions/
- Git Interview Questions and Answers for Developers, https://www.golinuxcloud.com/git-interview-questions-answers-developers-linux/
- Git Interview Questions, https://www.adaface.com/blog/git-interview-questions

Databases, indexing and queues:

- Database Indexing and Query Optimization, https://www.techinterview.org/post/3233463536/database-indexing-query-optimization/
- Database Indexing, https://prachub.com/resources/database-indexing
- Migration Interview Questions, https://interviewprep.org/migration-interview-questions/
- System Design: Job Scheduler, https://www.techinterview.org/post/3233461020/system-design-job-scheduler/
- System Design: Task Queue, https://www.techinterview.org/post/3233460795/system-design-task-queue/
- Spring Boot Interview Question: Sending Email Synchronously, https://javabulletin.substack.com/p/spring-boot-interview-question-sending

Retrieval and RAG:

- RAG Interview Questions, https://zerotomastery.io/blog/rag-interview-questions/
- RAG Evaluation Interview Questions, https://www.techinterview.org/post/3233477306/rag-evaluation-interview-questions/
- RAG System Design Interview, https://www.techinterview.org/post/3233476088/rag-system-design-interview/
- Vector Search System Design Questions, https://www.designgurus.io/system-design-interview/questions/vector-search
- RAG Interview Questions and Answers Hub, https://github.com/KalyanKS-NLP/RAG-Interview-Questions-and-Answers-Hub
- RAG Interview Questions for AI Engineers, https://schoolofcoreai.com/interview-prep/topics/rag-interview-questions

Design systems, accessibility and frontend state:

- Design System Interview Questions for Frontend Engineers: Tokens, APIs, Accessibility and Governance, https://prachub.com/resources/design-system-interview-questions-for-frontend-engineers-tokens-apis-accessibility-and-governance
- Accessibility Interview Questions, https://interviewprep.org/accessibility-interview-questions/
- Figma Interview Questions, https://algocademy.com/blog/?p=4333
- Explain what a single page app is and how to make one SEO-friendly, https://greatfrontend.com/questions/quiz/explain-what-a-single-page-app-is-and-how-to-make-one-seo-friendly
- React State Management Interview Questions, https://namastedev.com/topics/top-react-interview-questions/react-state-management-interview-questions
- React State Management: the 2026 Landscape, https://www.techinterview.org/post/3233474940/react-state-management-2026-landscape

A caution about the sources

Two of these pages contain answers that are wrong. One JWT page misstates where the public key sits and overstates tamper resistance. One email verification page has a comment claiming its error messages are identical for unknown and expired tokens while its own code returns a distinct expiry message, which leaks that the token was once valid.

Use these pages for the questions. Verify the answers against primary documentation before you believe them, and before you repeat them in an interview. That habit is the thing this course has been teaching since the first week, and it applies to interview preparation material as much as to anything an agent writes for you.